Cómo se ejecuta (detalle en el README.md)

- XAMPP Control Panel → Start en MySQL.
- HeidiSQL → abrir db/instalar_todo.sql → F9 (crea la base sitra con datos de prueba).
- VS Code → abrir la carpeta del repositorio → Run sobre SitraApplication.java (o mvn spring-boot:run). La API queda en http://localhost:8080.
- Postman → importar postman/coleccion-corte-vertical.json → Run collection (19 peticiones en orden).
- Pruebas automáticas: mvn test (unitarias, sin base de datos) y mvn test -Pintegracion (con MariaDB).

5.4 Pruebas de la arquitectura

Se entregan las dos modalidades que acepta la plantilla.

a) Colección Postman (postman/coleccion-corte-vertical.json) — 19 peticiones, todas con tests automáticos

#	Petición	Esperado	Qué valida
01	POST /auth/token (billetera)	200 + token	Autenticación con BCrypt
02	POST /auth/token con secreto incorrecto	401	AC-01
03	POST /transferencias sin token	401	AC-01: nunca llega al controlador
04	POST /transferencias 300.000	201 CONFIRMADA	CREATE por todas las capas
05	POST con el mismo código	409	Idempotencia (AC-02)
06	POST monto −50.000	400	Regla de negocio; queda RECHAZADO en bitácora
07	POST cuenta AH-9999	404	Lista blanca (mínimo privilegio)
08	GET /cuentas/AH-1001/saldo	200	HU-02, saldo del núcleo
09–10	GET lista y GET por id	200	READ: la creada aparece
11	PUT sobre la CONFIRMADA	409	ADR-004: no se maquilla lo que movió dinero
12	POST 5.000.000	422 RECHAZADA	Rechazo del núcleo
13	PUT de la RECHAZADA con 100.000	200 CONFIRMADA	UPDATE: corregir y reintentar
14–15	DELETE de la de 300.000 y GET por id	204 y 404	DELETE lógico con reverso
16–17	Token del auditor; bitácora con token de billetera	200 y 403	Autorización por rol
18	GET /auditoria?cuenta=AH-1001&desde=hoy&hasta=hoy	200	HU-03 y métrica X = A/B: creada, rechazos, corrección y anulación registrados
19	GET /auditoria/verificacion	200, integra = true	Nadie alteró ni borró filas

b) Pruebas JUnit 5 (src/test/java)

Clase de prueba	Tipo	Qué comprueba	Cómo se corre
JwtServicioTest (6 pruebas)	Unitaria	Token válido; token alterado (cambio de rol), con otro secreto, vencido o mal formado → rechazado; secreto corto → no arranca.	mvn test
TransferenciaServiceTest (11 pruebas)	Unitaria con Mockito	201/400/404/409/422/503 en CREATE; FALLIDA sin efecto parcial (AC-02); PUT 409 sobre CONFIRMADA y reintento de RECHAZADA; DELETE con reverso y 503 sin anular. Sin núcleo ni base de datos (AC-03).	mvn test
CorteVerticalIntegracionTest (4 pruebas)	Integración (MockMvc + MariaDB)	401 sin token; CRUD completo punta a punta; RECHAZADA → PUT → CONFIRMADA; saldo, 403 de la billetera en la bitácora y verificación íntegra.	mvn test -Pintegracion

Criterio de aceptación: desde una terminal limpia, siguiendo solo el README.md, se instala la base (F9 en HeidiSQL), se arranca la API y se ejecutan las pruebas (mvn test, mvn test -Pintegracion y el Runner de Postman) sin intervención del autor. Las capturas de esas ejecuciones se guardan en evidencias/entrega-07-pruebas-y-tradeoffs/.

SECCIÓN 6: EVALUACIÓN DE ARQUITECTURA Y TRADE-OFFS

6.1 Análisis de compromisos

Toda arquitectura sacrifica algo. Este es el balance de SITRA:

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	¿Por qué es aceptable en este proyecto?
Seguridad / trazabilidad (AC-01)	Eficiencia de desempeño (escrituras concurrentes)	Cada operación escribe una fila extra en la bitácora y el trigger lee la última huella. Medido en MariaDB: 6 inserciones simultáneas → cadena íntegra, pero 4 terminaron en deadlock (error 1213).	El volumen de una cooperativa es bajo; la operación que pierde el bloqueo se deshace completa y responde 503: la billetera reintenta con el mismo código sin duplicar (R-03).
Mantenibilidad (AC-03)	Rendimiento en consultas	JPA carga ClienteApi y Transferencia de cada fila de la bitácora (consultas N+1) al listarla.	La consulta del auditor siempre va por cuenta y fechas (índice idx_auditoria_cuenta_fecha); el volumen mantiene la meta de ≤ 5 s. Si crece: consulta nativa puntual.
Seguridad (AC-01)	Simplicidad / velocidad de desarrollo	Filtro JWT, firma HMAC y BCrypt agregan unas 240 líneas de código propio y un paso más en cada petición.	Se mueve dinero y datos personales (Ley 1581); se evita instalar un servidor de identidad (ADR-005) y el código está cubierto por 6 pruebas.
Integridad de la evidencia (AC-01)	Flexibilidad del CRUD	PUT solo sobre RECHAZADA o FALLIDA; DELETE exige reverso en el núcleo (ADR-004).	Una transferencia confirmada no puede "maquillarse": la bitácora siempre coincide con lo que pasó en el núcleo.
Cero presupuesto (RA-02) y cero personal (RA-03)	Disponibilidad	Todo corre en un solo equipo existente, sin réplicas ni balanceo (Sección 3.5).	La alta disponibilidad está fuera del alcance (1.1); lo crítico es no perder la bitácora: respaldo diario con mysqldump, que trae XAMPP.

6.2 Puntos de sensibilidad y riesgos técnicos

Un punto de sensibilidad es un elemento de la arquitectura cuyo cambio impacta directamente un atributo de calidad.

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Prob. / Impacto	Mitigación propuesta
R-01	Secreto de firma del token (SITRA_JWT_SECRET)	Seguridad	Si se filtra, cualquiera fabrica tokens y suplanta a la billetera o al auditor.	Baja / Crítico	Variable de entorno fuera de Git (.env en .gitignore); mínimo 32 caracteres; aleatorio si falta; token de 30 min; rotación antes de producción.
R-02	Orden "primero el núcleo, después guardar" en TransferenciaService	Fiabilidad / integridad	El núcleo descuenta pero SITRA falla al guardar: dinero movido sin transferencia registrada.	Baja / Alto	Con el simulador, la misma transacción lo deshace. Con el núcleo real: enviar el código de operación al núcleo y conciliar a diario

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Prob. / Impacto	Mitigación propuesta
					contra sus reportes; evaluar guardar PENDIENTE antes de llamar.
R-03	Trigger de huella encadenada (lee la última fila de la bitácora)	Fiabilidad / eficiencia	Con escrituras simultáneas, InnoDB bloquea y aparecen deadlocks (medido: 4 de 6).	Media si crece el volumen / Medio	ManejadorErrores responde 503 reintentable; la idempotencia evita duplicados; si el volumen crece: reintento automático o una sola cola de escritura.
R-04	Credenciales de la base (root sin contraseña en XAMPP)	Seguridad	Alguien con acceso a la base desactiva los triggers y altera la bitácora.	Media / Crítico	Usuario sitra_app sin DELETE ni TRIGGER (probado: error 1142); la verificación de huellas detecta cualquier cambio (probado: ALTERADO / ROTA).
R-05	Transporte HTTP en el puerto 8080	Seguridad (confidencialidad)	Token y datos viajan sin cifrar en la red.	Alta en laboratorio / Alto	HTTPS en Spring Boot con certificado gratuito antes de producción (deuda del ADR-005).
R-06	Protocolo del núcleo sin confirmar (NucleoLegadoConector)	Adecuación funcional	No poder integrar el núcleo real dentro de los 6 meses.	Media / Crítico	Conector aislado (ADR-001): solo cambia una clase; es la primera tarea con el cliente.
R-07	MariaDB 10.4 de XAMPP sin soporte	Seguridad	Vulnerabilidades sin parche.	Media / Medio	Actualizar a MariaDB 10.11 LTS; los scripts ya se ejecutaron en 10.11 sin cambios.

6.3 Conclusión arquitectónica

La arquitectura entregada satisface los tres escenarios de la Sección 1.3 con evidencia del corte vertical. AC-01: sin token se responde 401 antes de llegar a los controladores (filtro JWT, peticiones 02–03 y pruebas JUnit); el 100 % de las operaciones, incluidas las rechazadas, quedan en la bitácora porque la base de datos la escribe con triggers y el servicio registra los rechazos en una transacción propia (petición 18), y nadie puede modificarla sin que la verificación de huellas lo detecte (petición 19 y Paso 5 de la demo). AC-02: si el núcleo no responde, la transferencia queda FALLIDA con 503 y sin efecto parcial, se puede reintentar con el mismo código (PUT) y un código repetido nunca duplica la operación (409). AC-03: el núcleo se cambia con una variable (SITRA_NUCLEO_MODO) y las pruebas unitarias corren sin núcleo y sin base de datos.

Si el proyecto pasara a producción real, lo primero que evolucionaría es el conector real del núcleo, en cuanto el cliente confirme cómo expone sus datos, junto con la conciliación diaria (R-02 y R-06). Después, el endurecimiento del despliegue: HTTPS, el usuario sitra_app de mínimo privilegio, un secreto JWT definido y rotado, MariaDB 10.11 y respaldos automáticos (R-01, R-04, R-05 y R-07). Ninguno de esos cambios toca las capas de presentación ni de negocio: esa es la prueba de que la estructura elegida funciona.

Lista de chequeo de entrega

#	Entregable	¿Incluido?	Dónde
1	DAS completo (Secciones 1–6) en PDF dentro de docs/	Sí	docs/DAS.pdf y docs/DAS.docx (v1.0)
2	3 escenarios de calidad ISO 25010 con métricas	Sí	Sección 1.3 (AC-01, AC-02, AC-03)
3	5 vistas: Contexto, Contenedores, Clases, Secuencia, Despliegue	Sí	Sección 3 y docs/diagramas/
4	Mínimo 1 ADR en docs/adr/ con la plantilla oficial	Sí (5)	docs/adr/ADR-001 a ADR-005
5	Repositorio Git con ≥ 10 commits atómicos y .gitignore correcto	Sí	Historial con Conventional Commits; .gitignore excluye .env
6	Script SQL DDL con PK, FK, CHECK, UNIQUE y borrado lógico	Sí	db/migrations/V1 a V4 (Sección 5.2)
7	CRUD punta a punta ejecutable (Modelo–Controlador–Rutas–API)	Sí	Sección 5.3; src/main/java
8	Colección Postman o pruebas de integración pasando	Sí (ambas)	postman/ y src/test/java (Sección 5.4)
9	README.md que permite instalar y ejecutar sin ayuda del autor	Sí	README.md, sección "Instalar y ejecutar"
10	Análisis de trade-offs y ≥ 3 puntos de sensibilidad	Sí (5 y 7)	Secciones 6.1 y 6.2